

LLM-YARA

Structured LLM-Assisted YARA Rule Generation

Static-Only LLM Pipeline for Malware Family Detection

Yassir BOUSSETA

March 2026

Problem & Motivation

Manual YARA authoring does not scale to hundreds of malware families.

- MOTIF dataset contains **138 eligible families** (PE binaries, min 6 samples each)
- Existing automated approaches trade off precision for recall
- No standard audit trail for how rules are produced

Goal: A static-only pipeline that generates YARA rules from PE binaries -- no malware execution required.

Scope constraints:

- Static features only (strings, imports, PE headers)
- No sandbox, no dynamic analysis, no behavioral traces
- Results are reported from saved artifacts

Dataset & Split Discipline

MOTIF dataset -- 138 malware families, PE-only, minimum 6 samples per family.

Leakage-safe 4-way split:

Split	Purpose	Usage
train_target	Rule generation & train-hit gating	Generation
test_target	Final family detection evaluation	Evaluation only
benign_dev	FPR acceptance gate (60% of benign)	Gating
benign_test	Final false-positive evaluation	Evaluation only

Parameters: test_ratio=0.3 , benign_dev_ratio=0.6 , seed=42

benign_dev is used in feature ranking and acceptance gating -- never for metric reporting.

benign_test is held out until final evaluation.

Pipeline

```
index --> split --> extract --> select --> generate --> evaluate
```

Extract

- Strings, imports, PE metadata

Select

- Top-30 features per family
- TF-IDF + chi-squared
- `train_target` vs `train_other` + `benign_dev`

Generate

- Primary path: structured JSON
- Deterministic renderer
- Repair path may preserve YARA text
- Max 3 repairs

Accept

- Train-hit: ≥ 1 train sample
- Benign-dev FPR: ≤ 0.02

What Was Compared

Method	Status	Description
llmyara-llm	Primary	LLM-based structured generation with acceptance gates
topstrings	Reproduced	Deterministic top-frequency string baseline
apiary-static	Re-implemented	Import-discriminative baseline
autoyara-bicluster	Re-implemented	Byte n-gram biclustering baseline

All four methods were **evaluated on the same splits** with the same evaluation harness.

Not executed (paper-only references):

- YAMME -- cited only, not executed on this repo split
- PackGenome -- packer-focused, different threat model

Main Results (All 138 Families)

Method	Families OK	Mean F1	Mean TPR	Mean Ben. FPR	Mean Specificity
llmyara-llm	128 / 138	0.4240	0.4043	0.0000	0.9988
topstrings	138 / 138	0.4195	0.4797	0.0061	0.9924
apiary-static	138 / 138	0.1557	0.6999	0.0718	0.8638
autoyara-bicluster	138 / 138	0.2760	0.7965	0.1183	0.8180

- All-family metrics: rejected families contribute zero
- **llmyara-llm** : numerically highest F1 / specificity, but **128/138** coverage
- **topstrings** : full **138/138** coverage

Source: `artifacts/final_motif_audit/comparison_summary.json`

F1 vs topstrings: Statistical Significance

Paired randomization test (5000 iterations, family-level F1 pairs):

Comparison	Delta F1	p-value	Significant?
llmyara-llm vs topstrings	+0.0044	0.848	No
llmyara-llm vs apiary-static	+0.2683	0.0002	Yes
llmyara-llm vs autoyara-bicluster	+0.1480	0.0002	Yes

Interpretation:

- The +0.004 F1 lead over topstrings is **not statistically significant** ($p = 0.85$)
- The LLM pipeline is similar to topstrings on F1, not better
- The F1 difference vs apiary-static and autoyara-bicluster is significant

Source: `artifacts/final_motif_audit/significance_vs_primary.json`

TPR vs FPR Trade-off

Method	Mean TPR	Mean Ben. FPR	Mean Specificity	Profile
llmyara-llm	0.40	0.000	0.999	Low recall, zero FP
topstrings	0.48	0.006	0.992	Moderate recall, low FP
apiary-static	0.70	0.072	0.864	High recall, high FP
autoyara-bicluster	0.80	0.118	0.818	Highest recall, worst FP

Key insight: The LLM pipeline trades recall for specificity.

- Acceptance gates reject rules that fire on benign samples
- This lowers benign matches, but also lowers recall and coverage
- Byte-level baselines achieve much higher TPR but also much higher benign FPR

Coverage Gap & Failure Analysis

128 accepted / 10 rejected families (all exhausted max 3 repair attempts).

Reason	Count	Families
<code>train_target_hits_below_min</code>	7	carbanak, dridex, mosaicregressor, nazar, nymaim, phorpiex, smokeloader
<code>benign_dev_fpr_exceeded</code>	3	cerber, dharma, ryuk

Why these fail:

- The 7 "no hits" families lack stable static anchors -- strings/imports are too polymorphic or packed
- The 3 "FPR exceeded" families produce rules that also match benign PE files (common imports/strings)
- All 10 used all 3 repair attempts before rejection

This is the main limitation. topstrings covers all 138 families.

Example Accepted Rule

Source family: nanolocker | Final metrics: F1=1.0 , TPR=1.0 , FPR=0.0

```
import "pe"
rule nanolocker_candidate_01 {
  strings:
    $a1 = "inet_addr" ascii
  condition:
    uint16(0) == 0x5A4D
    and pe.imports("advapi32.dll", "abortsystemshutdowna")
    and 1 of them
}
```

- Selected strings become YARA string terms
- Import predicate adds a family-specific structural check
- Final rule stays simple enough to audit

Implementation & Reproducibility

- Complete repo workflow:

`index -> split -> extract -> select -> generate -> evaluate`

- Packaging:

`Dockerfile` , `docker-compose.yml` , `start.sh`

- CLI supports demo runs, full runs, baseline comparison, and audit export
- Reported metrics come from `artifacts/final_motif_audit/`
- Replay path exists as a check on saved outputs

Source: `README.md` , `start.sh` , `artifacts/final_motif_audit/`

Limitations & Takeaway

Limitations

- Static-only -- no dynamic or behavioral features
- Tied with topstrings on F1 ($p = 0.85$)
- Lower recall than byte-level baselines
- 10/138 families with no accepted rule
- Single LLM model (`gpt-5-mini`)
- No packer handling or evasion resistance

Takeaway

In this comparison, the LLM method gives:

- **Zero benign FPR** (0.000)
- **Highest specificity** (0.999)
- **128 / 138 accepted families**
- **Numerically highest mean F1 among executed methods**

The main trade-off is lower benign FPR versus lower recall and lower family coverage. The F1 difference vs

`topstrings` is not significant

Appendix

Appendix: Full Metric Table

Method	Fam. OK	Mean F1	Mean TPR	Mean FPR	Mean Spec.	Mean Compl.	Mean Off- Target	Rules
Ilmyara-Ilm	128/138	0.4240	0.4043	0.0000	0.9988	11.78	0.0012	128
topstrings	138/138	0.4195	0.4797	0.0061	0.9924	8.94	0.0076	138
apiary-static	138/138	0.1557	0.6999	0.0718	0.8638	8.51	0.1362	138
autoyara- bicluster	138/138	0.2760	0.7965	0.1183	0.8180	11.49	0.1820	138

Off-target rate = fraction of non-family malware samples that trigger the rule.

Appendix: Ok-Only Metrics (128 Accepted Families)

Metric	All 138	Ok-Only (128)
Mean F1	0.4240	0.4571
Mean TPR	0.4043	0.4359
Mean Ben. FPR	0.0000	0.0000
Mean Specificity	0.9988	0.9987

- Ok-only F1 is ~8% higher because the 10 zero-contribution families are excluded
- Headline results use all-family numbers, not ok-only numbers
- topstrings ok-only = all-family (138/138 accepted, no gap)

Appendix: Rule Complexity

Method	Mean Complexity	Mean Strings	Mean Clauses
Ilmyara-Ilm	11.78	9.57	2.21
topstrings	8.94	7.94	1.00
apiary-static	8.51	7.51	1.00
autoyara-bicluster	11.49	5.85	5.64

- Complexity = string count + condition clause count
- LLM rules use the most strings and more condition clauses than topstrings/apiary
- topstrings and apiary use a single condition clause (N of them)
- autoyara has many clauses but fewer strings (byte patterns)

Appendix: Rejected Family Details

Family	Reason	Repairs	Train Hits
carbanak	train_target_hits_below_min	3	0
dridex	train_target_hits_below_min	3	0
mosaicregressor	train_target_hits_below_min	3	0
nazar	train_target_hits_below_min	3	0
nymaim	train_target_hits_below_min	3	0
phorpiex	train_target_hits_below_min	3	0
smokeloader	train_target_hits_below_min	3	0
cerber	benign_dev_fpr_exceeded (0.022)	3	2
dharma	benign_dev_fpr_exceeded (0.022)	3	6
ryuk	benign_dev_fpr_exceeded (0.065)	3	15

All 7 "no hits" families: LLM-selected features do not survive in any train sample (likely packed/polymorphic).

Appendix: Annotated YARA Rule -- nanolocker (F1 = 1.0)

Source: deliverables/examples/nanolocker.yar

```
import "pe"

rule nanolocker_candidate_01 {
  meta:
    author = "automated-threat-detection"
    description = "High-signal substring candidates for Nanolocker"
    family = "nanolocker"
  strings:
    $a1 = "0a-%m]mm" ascii
    $a2 = "9j`mmlm" ascii
    $a3 = "mmm%m]mm" ascii
    $a4 = "mmmmmmmmmmmmmmmmh)m" ascii
    $a5 = "q,1oam" ascii
    $a6 = "|%m]mm" ascii
    $a7 = "inet_addr" ascii
    $a8 = "2.83+?! " ascii
    // ... 12 more strings (20 total)
  condition:
    uint16(0) == 0x5A4D
    and pe.imports("advapi32.dll", "abortsystemshutdowna")
    and 1 of them
```

Appendix: Pipeline Configuration

Parameter	Value	Section
seed	42	Splits / significance
test_ratio	0.3	Split
benign_dev_ratio	0.6	Split
min_family_size	6	Split
strings_min_len	6	Features
strings_max_per_file	150	Features
max_file_bytes	16 MB	Features
top_k_features	30	Selection
max_strings	20	YARA
max_rule_bytes	2048	YARA
max_repairs	3	YARA
benign_dev_fpr_threshold	0.02	YARA

Appendix: Replay Verification

Metric	Primary Run	Replay Run	Delta
Families OK	128	128	0
Mean F1	0.4240	0.4278	+0.0038
Mean TPR	0.4043	0.4043	0.0000
Mean Ben. FPR	0.0000	0.0000	0.0000
Mean Specificity	0.9988	0.9990	+0.0002
Mean Complexity	11.78	11.72	-0.06
Mean Strings	9.57	9.51	-0.06
Mean Clauses	2.21	2.21	0.00

- Replay uses cached LLM responses
- Mean F1 changes by 0.0038 while families OK and mean TPR stay the same
- Replay is a check on the saved outputs, not a claim of exact equality